

✓ Extração de Nome, RG e RA em Atestados de Matrícula

Notebook organizado para Google Colab com foco apenas em:

- Nome do aluno
- RG do aluno
- RA do aluno

Métodos utilizados:

- OCR com Tesseract
- Document Parsing com Donut

O resultado final é salvo em CSV e JSON.

```
# 1. Instalação das dependências
!pip -q install pdf2image pytesseract transformers torch pillow pandas sentencepiece
!apt-get -qq update
!apt-get -qq install -y poppler-utils tesseract-ocr tesseract-ocr-por
```

W: Skipping acquire of configured file 'main/source/Sources' as repository '<https://r2u.stat.illinois.edu/ubuntu> jammy InRe

```
# 2. Importações e configurações
import os
import re
import json
import pandas as pd
import pytesseract
import torch
```

```
from PIL import Image
from pdf2image import convert_from_path
from google.colab import files
from transformers import DonutProcessor, VisionEncoderDecoderModel
```

```
PASTA_SAIDA = "/content/saida_atestados"
PASTA_IMAGENS = os.path.join(PASTA_SAIDA, "imagens")
os.makedirs(PASTA_IMAGENS, exist_ok=True)
```

```
MODELO_DONUT = "naver-clova-ix/donut-base-finetuned-docvqa"
DEVICE = "cuda" if torch.cuda.is_available() else "cpu"
```

```
print("Dispositivo em uso:", DEVICE)
```

Dispositivo em uso: cpu

```
# 3. Upload dos PDFs
arquivos_enviados = files.upload()
pdfs = [nome for nome in arquivos_enviados.keys() if nome.lower().endswith(".pdf")]
```

```
if not pdfs:
    raise ValueError("Nenhum PDF foi enviado.")
```

```
print("Arquivos recebidos:")
for pdf in pdfs:
    print("-", pdf)
```

Escolher Arquivos 3 arquivos

```
e30b3a51-1cb8-4f2b-a316-3bd64d2e2638.pdf(application/pdf) - 61990 bytes, last modified: 18/03/2026 - 100% done
58e40860-51bd-4bc1-a0f9-155fd2a2d0e1.pdf(application/pdf) - 62237 bytes, last modified: 18/03/2026 - 100% done
ae19cfa2-ec17-4882-b641-01426ce4317a.pdf(application/pdf) - 61985 bytes, last modified: 18/03/2026 - 100% done
Saving e30b3a51-1cb8-4f2b-a316-3bd64d2e2638.pdf to e30b3a51-1cb8-4f2b-a316-3bd64d2e2638.pdf
Saving 58e40860-51bd-4bc1-a0f9-155fd2a2d0e1.pdf to 58e40860-51bd-4bc1-a0f9-155fd2a2d0e1.pdf
Saving ae19cfa2-ec17-4882-b641-01426ce4317a.pdf to ae19cfa2-ec17-4882-b641-01426ce4317a.pdf
Arquivos recebidos:
- e30b3a51-1cb8-4f2b-a316-3bd64d2e2638.pdf
- 58e40860-51bd-4bc1-a0f9-155fd2a2d0e1.pdf
- ae19cfa2-ec17-4882-b641-01426ce4317a.pdf
```

```
# 4. Funções de apoio
```

```
def pdf_para_imagem(caminho_pdf, pasta_imagens):
    paginas = convert_from_path(caminho_pdf)
    if not paginas:
        raise ValueError(f"Nenhuma página encontrada em {caminho_pdf}")
```

```

nome_base = os.path.splitext(os.path.basename(caminho_pdf))[0]
caminho_imagem = os.path.join(pasta_imagens, f"{nome_base}.png")

# Usa apenas a primeira página
paginas[0].save(caminho_imagem, "PNG")
return caminho_imagem

def extrair_texto_ocr(caminho_imagem):
    imagem = Image.open(caminho_imagem).convert("RGB")
    texto = pytesseract.image_to_string(imagem, lang="por")
    return texto

def normalizar_numero(valor):
    if valor is None:
        return None
    valor = re.sub(r"\D", "", valor)
    return valor if valor else None

def extrair_campos_por_regex(texto):
    texto_limpo = " ".join(texto.split())

    padrao_principal = re.search(
        r"atesta\s+que\s+(.*?),\s*RG\s*([\d\.\-\s]+),\s*RA\s*([\d\.\-\s]+)",
        texto_limpo,
        re.IGNORECASE
    )

    if padrao_principal:
        return {
            "nome_aluno": padrao_principal.group(1).strip(),
            "rg_aluno": normalizar_numero(padrao_principal.group(2)),
            "ra_aluno": normalizar_numero(padrao_principal.group(3))
        }

    nome = None
    rg = None
    ra = None

    nome_match = re.search(
        r"atesta\s+que\s+(.*?)(?:,\s*RG|\s+RG)",
        texto_limpo,
        re.IGNORECASE
    )
    if nome_match:
        nome = nome_match.group(1).strip()

    rg_match = re.search(r"\bRG\s*[: ]\s*([\d\.\-\s]+)", texto_limpo, re.IGNORECASE)
    if rg_match:
        rg = normalizar_numero(rg_match.group(1))

    ra_match = re.search(r"\bRA\s*[: ]\s*([\d\.\-\s]+)", texto_limpo, re.IGNORECASE)
    if ra_match:
        ra = normalizar_numero(ra_match.group(1))

    return {
        "nome_aluno": nome,
        "rg_aluno": rg,
        "ra_aluno": ra
    }
}

```

```

# 5. Carregamento do Donut e extração por pergunta
processor = DonutProcessor.from_pretrained(MODELO_DONUT)
model = VisionEncoderDecoderModel.from_pretrained(MODELO_DONUT).to(DEVICE)
model.eval()

def perguntar_donut(caminho_imagem, pergunta, max_length=64):
    imagem = Image.open(caminho_imagem).convert("RGB")

    prompt = f"<s_docvqa><s_question>{pergunta}</s_question><s_answer>"

    decoder_input_ids = processor.tokenizer(
        prompt,
        add_special_tokens=False,
        return_tensors="pt"
    ).input_ids.to(DEVICE)

    pixel_values = processor(imagem, return_tensors="pt").pixel_values.to(DEVICE)

    with torch.no_grad():

```

```

outputs = model.generate(
    pixel_values,
    decoder_input_ids=decoder_input_ids,
    max_length=max_length,
    pad_token_id=processor.tokenizer.pad_token_id,
    eos_token_id=processor.tokenizer.eos_token_id,
    use_cache=True,
    num_beams=1,
    bad_words_ids=[[processor.tokenizer.unk_token_id]],
    return_dict_in_generate=True
)

resposta = processor.batch_decode(outputs.sequences, skip_special_tokens=True)[0]
return resposta.strip()

def extrair_campos_donut(caminho_imagem):
    nome = perguntar_donut(caminho_imagem, "What is the full name of the student?")
    rg = perguntar_donut(caminho_imagem, "What is the student's RG number?")
    ra = perguntar_donut(caminho_imagem, "What is the student's RA number?")

    return {
        "nome_aluno_donut": nome if nome else None,
        "rg_aluno_donut": normalizar_numero(rg),
        "ra_aluno_donut": normalizar_numero(ra)
    }

```

Loading weights: 100% 484/484 [00:01<00:00, 396.90it/s, Materializing param=encoder.encoder.layers.3.blocks.1.output.dense.weight]
 The tied weights mapping and config for this model specifies to tie decoder.model.decoder.embed_tokens.weight to decoder.lm_

```

# 6. Processamento dos arquivos
resultados = []

for pdf in pdfs:
    print(f"Processando: {pdf}")
    caminho_pdf = f"/content/{pdf}"

    try:
        caminho_imagem = pdf_para_imagem(caminho_pdf, PASTA_IMAGENS)
        texto_ocr = extrair_texto_ocr(caminho_imagem)

        campos_ocr = extrair_campos_por_regex(texto_ocr)
        campos_donut = extrair_campos_donut(caminho_imagem)

        # Resultado final: prioriza OCR e usa Donut como apoio quando OCR não encontrar
        resultado_final = {
            "arquivo": pdf,
            "nome_aluno": campos_ocr["nome_aluno"] or campos_donut["nome_aluno_donut"],
            "rg_aluno": campos_ocr["rg_aluno"] or campos_donut["rg_aluno_donut"],
            "ra_aluno": campos_ocr["ra_aluno"] or campos_donut["ra_aluno_donut"],
            "nome_aluno_ocr": campos_ocr["nome_aluno"],
            "rg_aluno_ocr": campos_ocr["rg_aluno"],
            "ra_aluno_ocr": campos_ocr["ra_aluno"],
            "nome_aluno_donut": campos_donut["nome_aluno_donut"],
            "rg_aluno_donut": campos_donut["rg_aluno_donut"],
            "ra_aluno_donut": campos_donut["ra_aluno_donut"],
        }

        resultados.append(resultado_final)

    except Exception as e:
        resultados.append({
            "arquivo": pdf,
            "nome_aluno": None,
            "rg_aluno": None,
            "ra_aluno": None,
            "nome_aluno_ocr": None,
            "rg_aluno_ocr": None,
            "ra_aluno_ocr": None,
            "nome_aluno_donut": None,
            "rg_aluno_donut": None,
            "ra_aluno_donut": None,
            "erro": str(e)
        })

df_resultados = pd.DataFrame(resultados)
df_resultados

```

Processando: e30b3a51-1cb8-4f2b-a316-3bd64d2e2638.pdf
Processando: 58e40860-51bd-4bc1-a0f9-155fd2a2d0e1.pdf
Processando: ae19cfa2-ec17-4882-b641-01426ce4317a.pdf

	arquivo	nome_aluno	rg_aluno	ra_aluno	nome_aluno_ocr	rg_aluno_ocr	ra_aluno_ocr	nome_aluno_donut	rg_aluno_
0	e30b3a51-1cb8-4f2b-a316-3bd64d2e2638.pdf	JOÃO PEDRO DA SILVEIRA	500664699	24010951	JOÃO PEDRO DA SILVEIRA	500664699	24010951	What is the full name of the student? patricula	5006
1	58e40860-51bd-4bc1-a0f9-155fd2a2d0e1.pdf	GABRIEL DOS SANTOS	608332975	24013743	GABRIEL DOS SANTOS	608332975	24013743	What is the full name of the student? patricula	6083
2	ae19cfa2-ec17-4882-b641-01426ce4317a.pdf	ÍTALO DE SOUZA	537383669	24007472	ÍTALO DE SOUZA	537383669	24007472	What is the full name of the student? nutritic...	5373

Próximas etapas: [Gerar código com df_resultados](#) [New interactive sheet](#)

```
# 7. Exportação dos resultados
caminho_csv = os.path.join(PASTA_SAIDA, "resultado_atestados.csv")
caminho_json = os.path.join(PASTA_SAIDA, "resultado_atestados.json")

df_resultados.to_csv(caminho_csv, index=False, encoding="utf-8-sig")

with open(caminho_json, "w", encoding="utf-8") as arquivo_json:
    json.dump(resultados, arquivo_json, ensure_ascii=False, indent=4)

print("Arquivos gerados:")
print("-", caminho_csv)
print("-", caminho_json)

files.download(caminho_csv)
files.download(caminho_json)
```

Arquivos gerados:
- /content/saida_atestados/resultado_atestados.csv
- /content/saida_atestados/resultado_atestados.json